1. Technische Schritte und Entscheidungen

Initialer Aufbau

Das Angular-Projekt wurde mit dem Angular CLI erstellt. Es wurde sich für **Standalone-Components** entschieden, da dies dem modernen Angular-Standard entspricht. Die Projektstruktur wurde nach Features gegliedert, um die Wartbarkeit zu gewährleisten.

Das Backend wurde für diese Abgabe nicht implementiert. Es existiert lediglich ein leeres Spring-Boot-Gerüst (ServerApplication-Klasse), das für die finale Abgabe vorgesehen ist. Die Daten werden daher ausschließlich lokal in der AppComponent als hardgecodete Beispieldaten gehalten.

Komponenten-Architektur

Eine **komponentenbasierte Architektur** mit folgender Hierarchie wurde umgesetzt:

- **AppComponent** – fungiert als zentrale Orchestrator-Komponente. Sie hält die Liste der Touren und die aktuell ausgewählte Tour. Der State wird hier verwaltet und über @Input() an Child-Komponenten weitergegeben. Events kommen über @Output() zurück.
- **TourListComponent** – zuständig für die Anzeige aller Touren und das Emittieren von Auswahl-Events.
- **TourDetailComponent** – zeigt die Details einer ausgewählten Tour an, inklusive Karten-Placeholder und der Tour-Logs-Komponente.
- **TourFormComponent** – verwaltet das Erstellen und Bearbeiten von Touren mit Validierung.
- **TourLogsComponent** – ermöglicht CRUD-Operationen für Tour-Logs einer ausgewählten Tour.
- **ActionButtonComponent** – eine wiederverwendbare UI-Komponente für Buttons.

State-Management

Für die Zwischenabgabe wurde ein einfacher Ansatz gewählt: Alle Daten werden in der AppComponent als lokale Arrays gespeichert. Der TourService wurde zwar angelegt, aber bewusst leer gelassen, da für diesen Schritt keine Backend-Anbindung erforderlich war. Dadurch konnte der Fokus auf die Frontend-Features gelegt werden.

Datenfluss

Der Datenfluss verläuft unidirektional von der Parent- zur Child-Komponente über @Input(). Nutzerinteraktionen (wie das Auswählen einer Tour, Speichern eines Formulars oder Löschen eines Logs) lösen über @Output() Events aus, die in der AppComponent landen. Dort wird der State aktualisiert und anschließend wieder nach unten propagiert. Dadurch bleibt die Logik zentral und vorhersehbar.

Formular-Handling

Für die Formulare wurden **Template-driven Forms** mit ngModel verwendet. Die Validierung erfolgt über HTML5-Attribute (required, min) in Kombination mit der Angular-Formularvalidierung. Der Submit-Button ist deaktiviert, solange das Formular ungültig ist – dies verhindert, dass inkonsistente Daten gespeichert werden.

Karten-Integration

Für die Karten-Placeholder wurde **Leaflet** eingebunden. Die Karte initialisiert mit einer Standard-Ansicht (Wien) und passt ihre Größe an, wenn die Komponente geladen wird oder zwischen Touren gewechselt wird. Dies war notwendig, weil Leaflet manchmal nicht korrekt rendert, wenn der Container beim Initialisieren noch nicht sichtbar ist.

2. Probleme und gewählte Lösungen

Problem: Leaflet-Karte rendert nicht korrekt

Problem: Beim Wechsel zwischen Touren wurde die Karte manchmal nicht richtig dargestellt oder die Größe stimmte nicht. Dies lag daran, dass der Container beim Initialisieren der Karte noch nicht vollständig sichtbar war.

Lösung: Die initiale Kartenerstellung wurde in ngAfterViewInit() gesetzt und ein setTimeout eingebaut, um map.invalidateSize() nach einer kurzen Verzögerung aufzurufen. Für Tour-Wechsel wird ngOnChanges() genutzt, um zu erkennen, wenn sich die Tour ändert, und invalidateSize() erneut aufgerufen. Zusätzlich wurde in ngOnDestroy() eine Bereinigung eingebaut, um die Karten-Instanz zu entfernen und Memory-Leaks zu vermeiden.

Problem: Daten-Synchronisation zwischen Tour und Logs

Problem: Wenn Logs hinzugefügt, bearbeitet oder gelöscht wurden, mussten die Änderungen im State der Parent-Komponente ankommen. Ohne saubere Event-Weitergabe wäre die UI inkonsistent geworden.

Lösung: Es wurde eine Event-Kette umgesetzt: TourLogsComponent emittiert logsChange → TourDetailComponent emittiert tourUpdated → AppComponent aktualisiert das Touren-Array und setzt die aktuell ausgewählte Tour neu. So bleibt die Datenkonsistenz über alle Komponenten hinweg erhalten.

Problem: Formular-Status beim Abbrechen

Problem: Beim Abbrechen des Tour-Formulars blieben die eingegebenen Daten erhalten, was verwirrend war.

Lösung: Eine resetLogForm()-Methode wurde implementiert, die beim Abbrechen das tourForm-Objekt auf eine leere Tour zurücksetzt. In ngOnChanges() wird sichergestellt, dass beim Bearbeiten das Formular mit den vorhandenen Daten befüllt wird und beim Erstellen einer neuen Tour das Formular geleert wird.

3. UML Use Case Diagramm

Die folgenden Anwendungsfälle sind in der aktuellen Frontend-Version umgesetzt:

Touren verwalten

- *Tour anlegen* – Das Formular wird mit Tour-Attributen ausgefüllt (Name, Beschreibung, Startort, Zielort, Transportmittel, Distanz, geschätzte Zeit, Bild-URL)
- *Tour anzeigen* – Eine Tour wird aus der Liste ausgewählt, die Details werden angezeigt
- *Tour bearbeiten* – Vorhandene Tour-Attribute werden geändert
- *Tour löschen* – Eine Tour wird aus dem System entfernt

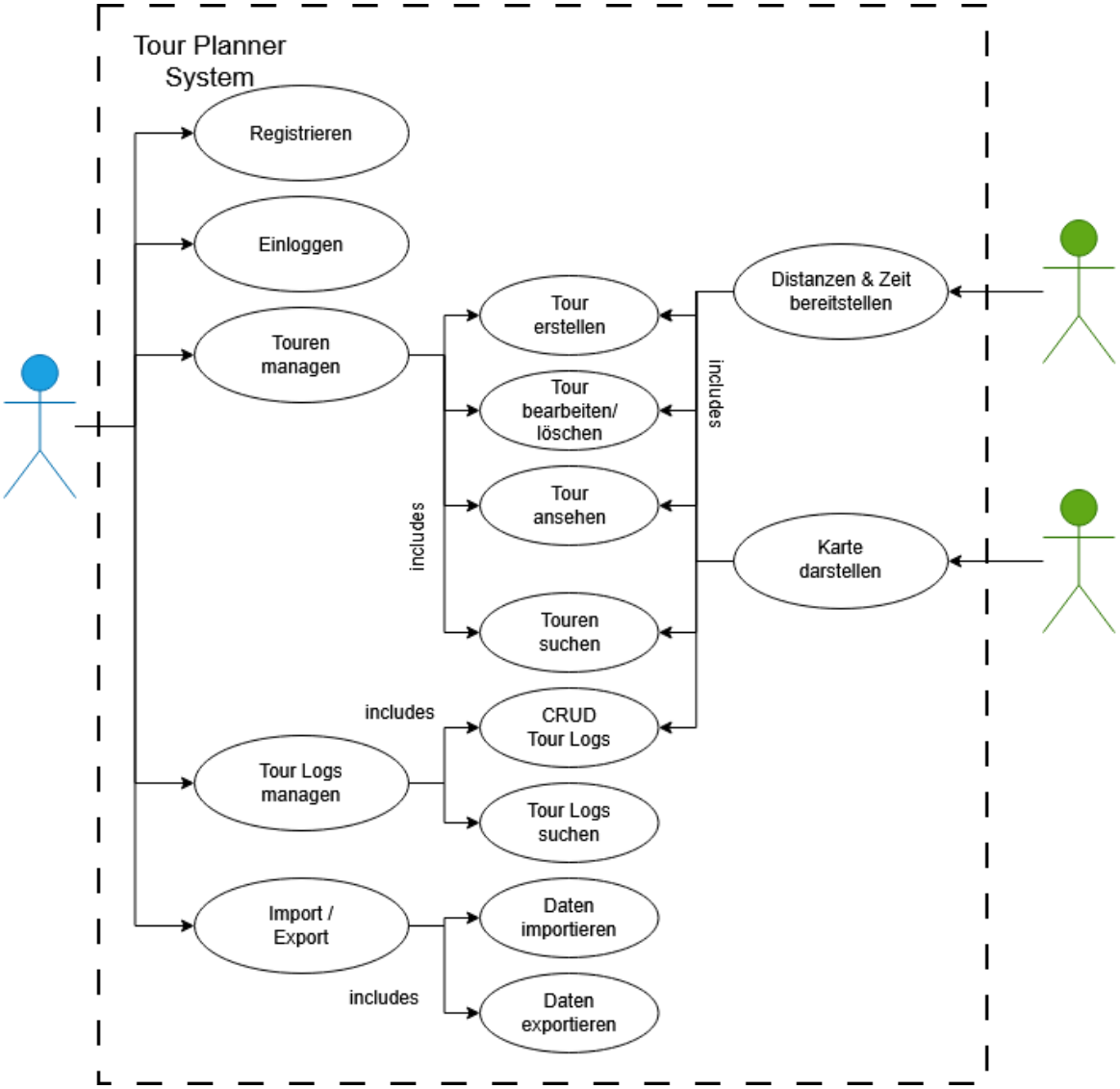
Tour-Logs verwalten

- *Log anlegen* – Für eine ausgewählte Tour wird ein Log-Eintrag hinzugefügt (Datum/Uhrzeit, Kommentar, Schwierigkeit, Distanz, Zeit, Bewertung)
- *Logs anzeigen* – Alle Logs einer Tour werden als Liste angezeigt
- *Log bearbeiten* – Ein vorhandener Log-Eintrag wird geändert
- *Log löschen* – Ein Log-Eintrag wird entfernt

Karte anzeigen – Beim Anzeigen einer Tour wird eine Karten-Placeholder dargestellt

Login/Registrieren – Buttons sind vorhanden, aber noch nicht implementiert (geplant für zukünftige Iteration)

Akteur: Nutzer (einziger Akteur im aktuellen Scope)



4. UI-Flow / Wireframes

Die Wireframes werden separat hinzugefügt. Der UI-Flow folgt dieser Struktur:

1. Hauptbildschirm

- Linke Seitenleiste: Tour-Liste mit Thumbnails, Namen und Kurzinformationen
- Rechter Inhaltsbereich: Tour-Details oder leerer Zustand, wenn keine Tour ausgewählt ist
- Header: Branding und Authentifizierungs-Buttons (Login/Register)

2. Tour anlegen

- Der "Create Tour"-Button in der Tour-Liste wird geklickt
- Das Formular erscheint im Inhaltsbereich mit Feldern für alle Tour-Attribute
- Das Formular wird ausgefüllt (Validierung stellt sicher, dass Pflichtfelder ausgefüllt sind)

- Durch Klick auf "Save" wird die Tour zur Liste hinzugefügt und deren Details werden angezeigt

3. Tour bearbeiten

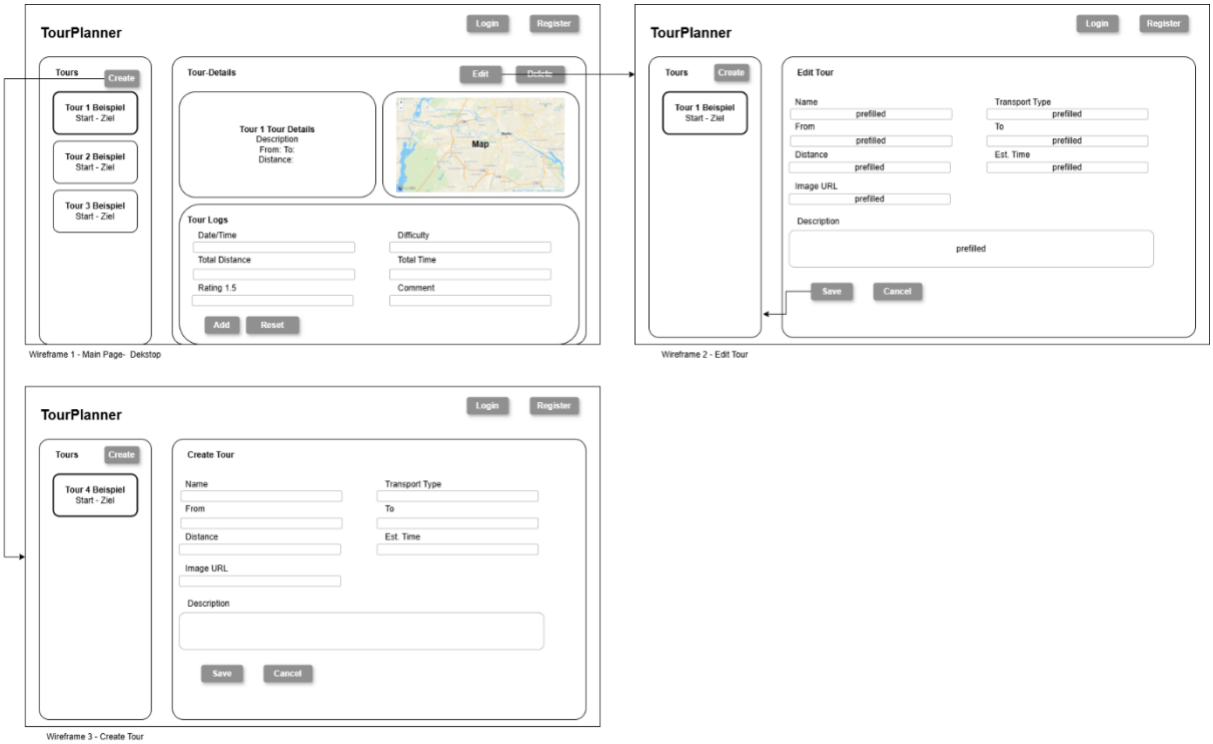
- Eine Tour wird ausgewählt, anschließend wird im Detailbereich auf "Edit Tour" geklickt
- Das Formular erscheint mit den vorhandenen Daten vorausgefüllt
- Die Felder werden geändert und gespeichert

4. Tour-Logs verwalten

- Unterhalb der Tour-Details werden vorhandene Logs als Karten angezeigt
- Ein Formular zum Hinzufügen/Bearbeiten von Logs ist direkt in den Logs-Bereich integriert
- Neue Logs können hinzugefügt, bestehende bearbeitet oder gelöscht werden

5. Leerzustände

- Wenn keine Touren existieren, zeigt der Inhaltsbereich "Please create or select a tour"
- Wenn für eine Tour keine Logs existieren, wird eine entsprechende Meldung angezeigt



Link: https://drive.google.com/file/d/1pCHk_cqE9M6VAqspxQEabE_dlFUIQDn5/view?usp=sharing

5. Anwendungsarchitektur – UML Klassendiagramm (textuelle Beschreibung)

Hauptklassen / Komponenten:

AppComponent

- Attribute: tours: Tour[], selectedTour: Tour | null, showTourForm: boolean, editMode: boolean
- Methoden: selectTour(), openCreateTourForm(), openEditTourForm(), saveTour(), deleteTour(), updateTourState(), cancelTourForm()

TourListComponent

- Inputs: tours: Tour[], selectedTourId: number
- Outputs: tourSelected, createTourClicked

TourDetailComponent

- Input: tour: Tour
- Outputs: editTour, deleteTour, tourUpdated
- Interne Methoden: initMap(), updateLogs()

TourFormComponent

- Inputs: tour: Tour | null, editMode: boolean
- Outputs: save, cancel
- Interne Methoden: createEmptyTour()

TourLogsComponent

- Input: logs: TourLog[]
- Output: logsChange
- Interne Attribute: logForm: TourLog, editingLogId: number
- Interne Methoden: saveLog(), editLog(), deleteLog(), resetLogForm()

ActionButtonComponent

- Inputs: label, type, variant, disabled
- Output: buttonClick

TourService (geplant für zukünftige Backend-Integration)

- Derzeit leer, soll später HTTP-Requests übernehmen

Beziehungen:

- AppComponent aggregiert alle anderen Komponenten
- TourDetailComponent enthält TourLogsComponent

- Alle Komponenten verwenden ActionButtonComponent als wiederverwendbares UI-Element
- Tour und TourLog sind Interfaces mit einer 1:n-Beziehung (eine Tour hat mehrere Logs)

6. Unit-Testing-Strategie

Begründung der Testauswahl

Die Tests wurden auf kritische Pfade fokussiert, die zum Absturz der Anwendung oder zu inkonsistentem Verhalten führen könnten. Die .spec.ts-Dateien enthalten derzeit einfache "should create"-Tests. Für eine vollständige Testsuite würde die Abdeckung auf folgende Bereiche erweitert werden:

Kritischer Code, der getestet werden sollte

Komponente / Methode	Begründung
saveTour() in AppComponent	Diese Methode modifiziert das zentrale Daten-Array. Ein Fehler könnte zu Datenverlust oder doppelten Touren führen.
deleteTour() in AppComponent	Permanentes Entfernen von Daten – es muss sichergestellt sein, dass die richtige Tour gelöscht wird und die UI sich korrekt aktualisiert.
TourLogsComponent.saveLog()	Behandelt sowohl das Anlegen als auch das Aktualisieren von Logs. Falsche Logik könnte Log-Daten beschädigen.
Formular-Validierung in TourFormComponent	Verhindert, dass ungültige Daten ins System gelangen. Fehlende Validierung könnte zu Laufzeitfehlern führen.
Leaflet-Karten-Lebenszyklus (ngAfterViewInit, ngOnDestroy)	Memory-Leaks könnten auftreten, wenn die Karten-Instanz nicht ordnungsgemäß bereinigt wird.

Gewählter Testansatz

Jasmine wird für Unit-Tests verwendet, die **Angular Testing Utilities** für Komponenten-Integrationstests. Zentrale Testfälle, die implementiert werden würden:

- **AppComponent**: Es wird geprüft, dass das Hinzufügen einer Tour die Länge des Touren-Arrays erhöht; es wird geprüft, dass das Löschen das korrekte Element entfernt.

- **TourFormComponent**: Es wird getestet, dass ein ungültiges Formular den Submit-Button deaktiviert; es wird getestet, dass das Formular bei Gültigkeit das save-Event mit korrekten Daten auslöst.
- **TourLogsComponent**: Es wird getestet, dass das Hinzufügen eines Logs logsChange mit aktualisiertem Array auslöst; es wird getestet, dass das Bearbeiten den richtigen Log aktualisiert.
- **ActionButtonComponent**: Es wird getestet, dass Button-Klick das Event auslöst; es wird getestet, dass deaktivierter Button kein Event auslöst.